

exploratory_analysis_not_quite_final

April 29, 2026

1 Milestone 1 Data Analysis

1.1 Comparison of Optimization Strategies for Logistic Regression

This notebook supports the Milestone report for the capstone project. It performs:

- basic exploratory data analysis on the Iris dataset
- data preprocessing and standardization
- training with BGD, SGD, and MBGD
- comparison of convergence, accuracy, and computational cost
- a small learning-rate sensitivity experiment

1.2 1. Imports

```
[1]: import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Make sure the project root is importable when this notebook is run from /
↳ notebooks
PROJECT_ROOT = Path.cwd().resolve().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

from train import train_model
from evaluation import compare_runs
from plots import plot_loss_curves, plot_accuracy_curves

plt.rcParams['figure.figsize'] = (8, 5)
```

1.3 2. Load the Diabetes Database

Attribute Information: - age: age in years - sex: - bmi: body mass index - bp: average blood pressure - s1: tc, total serum cholesterol - s2: ldl, low-density lipoproteins - s3: hdl, high-density lipoproteins - s4: tch, total cholesterol / HDL - s5: ltg, possibly log of serum triglycerides level - s6: glu, blood sugar level

Note: Each of these 10 feature variables have been mean centered and scaled by the standard deviation times the square root of `n_samples` (i.e. the sum of squares of each column totals 1).

```
[2]: diabetes = load_diabetes(as_frame=True, scaled=False)
df = diabetes.frame.copy().round(3)

df.head()
```

```
[2]:    age  sex  bmi   bp   s1   s2   s3   s4   s5   s6  target
0  59.0  2.0  32.1 101.0 157.0  93.2  38.0  4.0  4.860  87.0  151.0
1  48.0  1.0  21.6  87.0 183.0 103.2  70.0  3.0  3.892  69.0   75.0
2  72.0  2.0  30.5  93.0 156.0  93.6  41.0  4.0  4.673  85.0  141.0
3  24.0  1.0  25.3  84.0 198.0 131.4  40.0  5.0  4.890  89.0  206.0
4  50.0  1.0  23.0 101.0 192.0 125.4  52.0  4.0  4.290  80.0  135.0
```

1.4 3. Basic dataset inspection

```
[3]: print('Shape:', df.shape)
print('\nColumn names:')
print(df.columns.tolist())
print('\nMissing values per column:')
print(df.isnull().sum())
```

Shape: (442, 11)

Column names:

['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6', 'target']

Missing values per column:

```
age      0
sex      0
bmi      0
bp       0
s1       0
s2       0
s3       0
s4       0
s5       0
s6       0
target   0
dtype: int64
```

1.5 4. Summary statistics

```
[4]: df.describe()
```

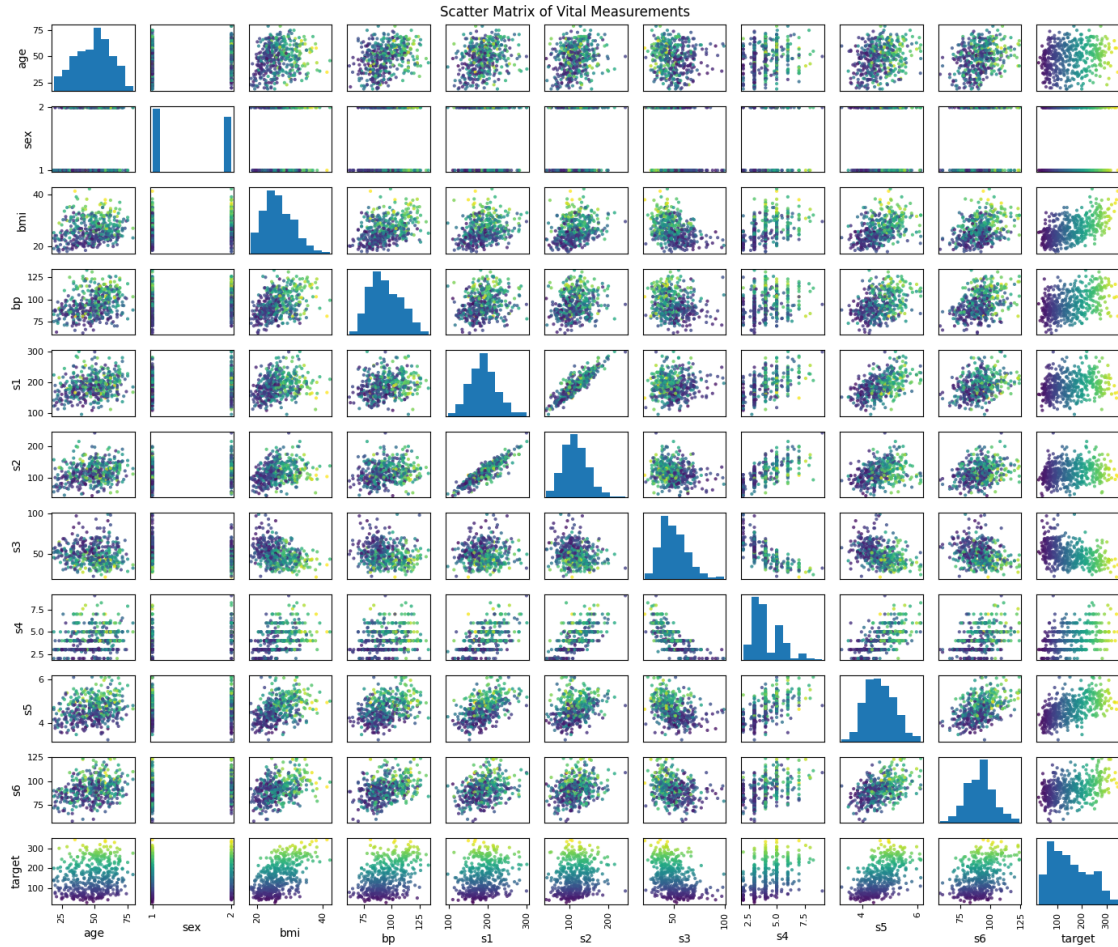
```
[4]:
```

	age	sex	bmi	bp	s1	s2 \
count	442.000000	442.000000	442.000000	442.000000	442.000000	442.000000
mean	48.518100	1.468326	26.375792	94.647014	189.140271	115.439140
std	13.109028	0.499561	4.418122	13.831283	34.608052	30.413081
min	19.000000	1.000000	18.000000	62.000000	97.000000	41.600000
25%	38.250000	1.000000	23.200000	84.000000	164.250000	96.050000
50%	50.000000	1.000000	25.700000	93.000000	186.000000	113.000000
75%	59.000000	2.000000	29.275000	105.000000	209.750000	134.500000
max	79.000000	2.000000	42.200000	133.000000	301.000000	242.400000

	s3	s4	s5	s6	target
count	442.000000	442.000000	442.000000	442.000000	442.000000
mean	49.788462	4.070249	4.641425	91.260181	152.133484
std	12.934202	1.290450	0.522394	11.496335	77.093005
min	22.000000	2.000000	3.258000	58.000000	25.000000
25%	40.250000	3.000000	4.277000	83.250000	87.000000
50%	48.000000	4.000000	4.620000	91.000000	140.500000
75%	57.750000	5.000000	4.997000	98.000000	211.500000
max	99.000000	9.090000	6.107000	124.000000	346.000000

1.6 5. Pairwise visualization

```
[5]: pd.plotting.scatter_matrix(  
    df.iloc[:, :],  
    c=df['target'], # Color by target variable  
    figsize=(14, 12),  
    diagonal='hist',  
    alpha=0.8,  
)  
plt.suptitle('Scatter Matrix of Vital Measurements')  
plt.tight_layout()  
plt.show()
```



1.7 6. Correlation analysis

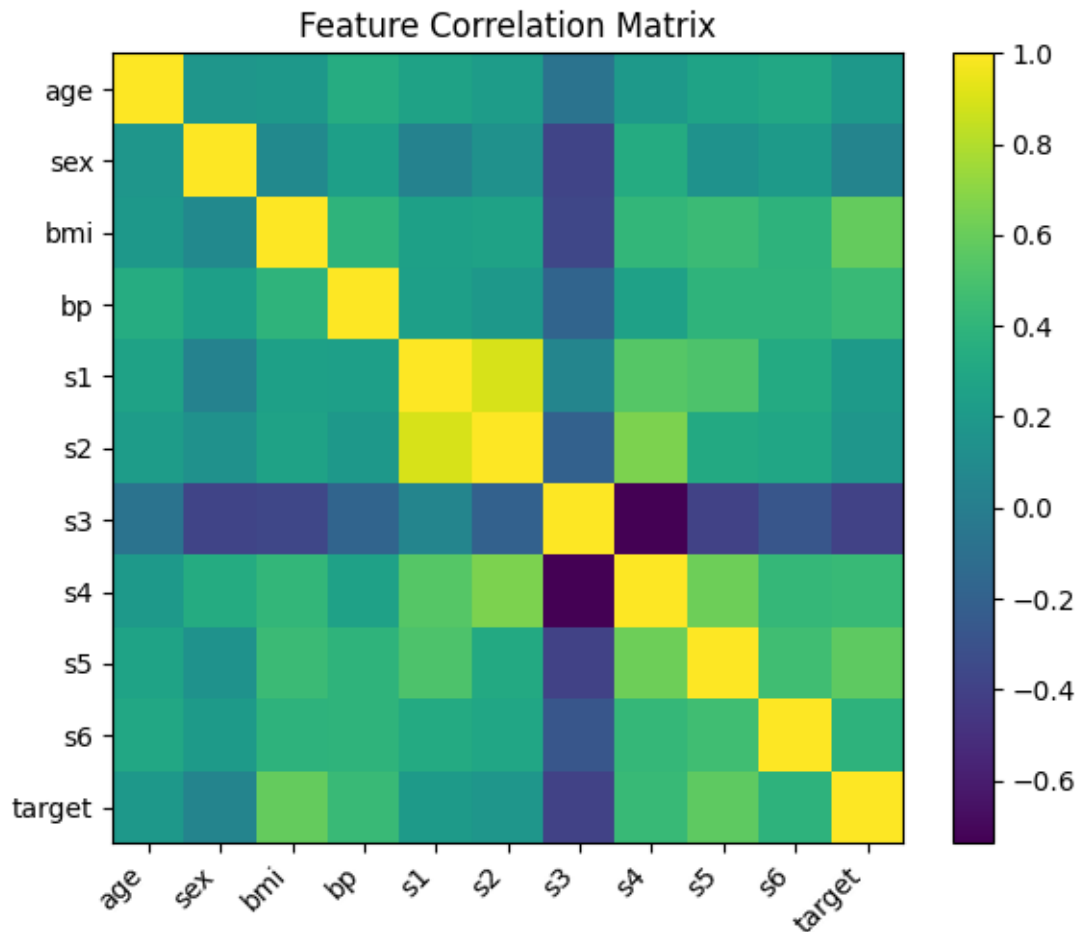
```
[6]: corr = df.iloc[:, :].corr()
corr
```

```
[6]:
```

	age	sex	bmi	bp	s1	s2	s3	\
age	1.000000	0.173737	0.185085	0.335428	0.260061	0.219243	-0.075181	
sex	0.173737	1.000000	0.088161	0.241010	0.035277	0.142637	-0.379090	
bmi	0.185085	0.088161	1.000000	0.395411	0.249777	0.261170	-0.366811	
bp	0.335428	0.241010	0.395411	1.000000	0.242464	0.185548	-0.178762	
s1	0.260061	0.035277	0.249777	0.242464	1.000000	0.896663	0.051519	
s2	0.219243	0.142637	0.261170	0.185548	0.896663	1.000000	-0.196455	
s3	-0.075181	-0.379090	-0.366811	-0.178762	0.051519	-0.196455	1.000000	
s4	0.203841	0.332115	0.413807	0.257650	0.542207	0.659817	-0.738493	
s5	0.270800	0.149913	0.446165	0.393489	0.515508	0.318353	-0.398554	
s6	0.301731	0.208133	0.388680	0.390430	0.325717	0.290600	-0.273697	
target	0.187889	0.043062	0.586450	0.441482	0.212022	0.174054	-0.394789	

	s4	s5	s6	target
age	0.203841	0.270800	0.301731	0.187889
sex	0.332115	0.149913	0.208133	0.043062
bmi	0.413807	0.446165	0.388680	0.586450
bp	0.257650	0.393489	0.390430	0.441482
s1	0.542207	0.515508	0.325717	0.212022
s2	0.659817	0.318353	0.290600	0.174054
s3	-0.738493	-0.398554	-0.273697	-0.394789
s4	1.000000	0.617836	0.417212	0.430453
s5	0.617836	1.000000	0.464664	0.565845
s6	0.417212	0.464664	1.000000	0.382483
target	0.430453	0.565845	0.382483	1.000000

```
[7]: plt.figure(figsize=(6, 5))
plt.imshow(corr, interpolation='nearest')
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha='right')
plt.yticks(range(len(corr.index)), corr.index)
plt.colorbar()
plt.title('Feature Correlation Matrix')
plt.tight_layout()
plt.show()
```



1.8 7. Load standardized train/test data

```
[8]: def load_data(test_size=0.2, random_state=42):
    diabetes = load_diabetes(scaled=False, as_frame=True)
    X = diabetes.data

    # Convert continuous target to binary classes
    # 1 if above median, 0 if below
    y = (diabetes.target > diabetes.target.median()).astype(int).values

    # Now stratification will work perfectly because there are only 2 classes
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
```

```

X_test = scaler.transform(X_test)

return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = load_data(test_size=0.2, random_state=42)

print('Training shape:', X_train.shape)
print('Test shape:', X_test.shape)
print('Training class counts:', np.bincount(y_train))
print('Test class counts:', np.bincount(y_test))

```

```

Training shape: (353, 10)
Test shape: (89, 10)
Training class counts: [176 177]
Test class counts: [45 44]

```

1.9 8. Main experiment: BGD vs SGD vs MBGD

```

[9]: seeds = [0, 1, 2, 3, 4] # repeat trials with these random seeds

settings = [
    {"optimizer": "bgd", "learning_rate": 0.05, "epochs": 400, "batch_size": 16},
    {"optimizer": "sgd", "learning_rate": 0.05, "epochs": 400, "batch_size": 16},
    {"optimizer": "mbgd", "learning_rate": 0.05, "epochs": 400, "batch_size": 16},
]

results = []
last_histories = {}

for seed in seeds:
    # ensure reproducible split / RNG for each trial
    np.random.seed(seed)

    X_train, X_test, y_train, y_test = load_data(test_size=0.2, random_state=seed)

    for cfg in settings:
        _, history = train_model(
            X_train,
            y_train,
            X_test,
            y_test,
            optimizer=cfg["optimizer"],
            learning_rate=cfg["learning_rate"],

```

```

        epochs=cfg["epochs"],
        batch_size=cfg["batch_size"],
    )

    results.append(
        {
            "seed": seed,
            "optimizer": cfg["optimizer"],
            "final_train_loss": history["train_loss"][-1],
            "final_test_loss": history["test_loss"][-1],
            "final_train_acc": history["train_acc"][-1],
            "final_test_acc": history["test_acc"][-1],
            "avg_epoch_time": float(np.mean(history.get("epoch_time",
↪[0]))),
        }
    )

    # keep last history per optimizer for quick plotting
    last_histories[cfg["optimizer"]] = history

df = pd.DataFrame(results)
summary_df = (
    df.groupby("optimizer")
    .agg(
        mean_final_test_acc=("final_test_acc", "mean"),
        std_final_test_acc=("final_test_acc", "std"),
        mean_final_test_loss=("final_test_loss", "mean"),
        std_final_test_loss=("final_test_loss", "std"),
        mean_epoch_time=("avg_epoch_time", "mean"),
    )
    .reset_index()
)

print("\nRepeated-trials summary (mean ± std over seeds):\n")
print(summary_df)

# plot curves for the last seed run (one per optimizer)
histories = [last_histories[cfg["optimizer"]] for cfg in settings]

```

Repeated-trials summary (mean ± std over seeds):

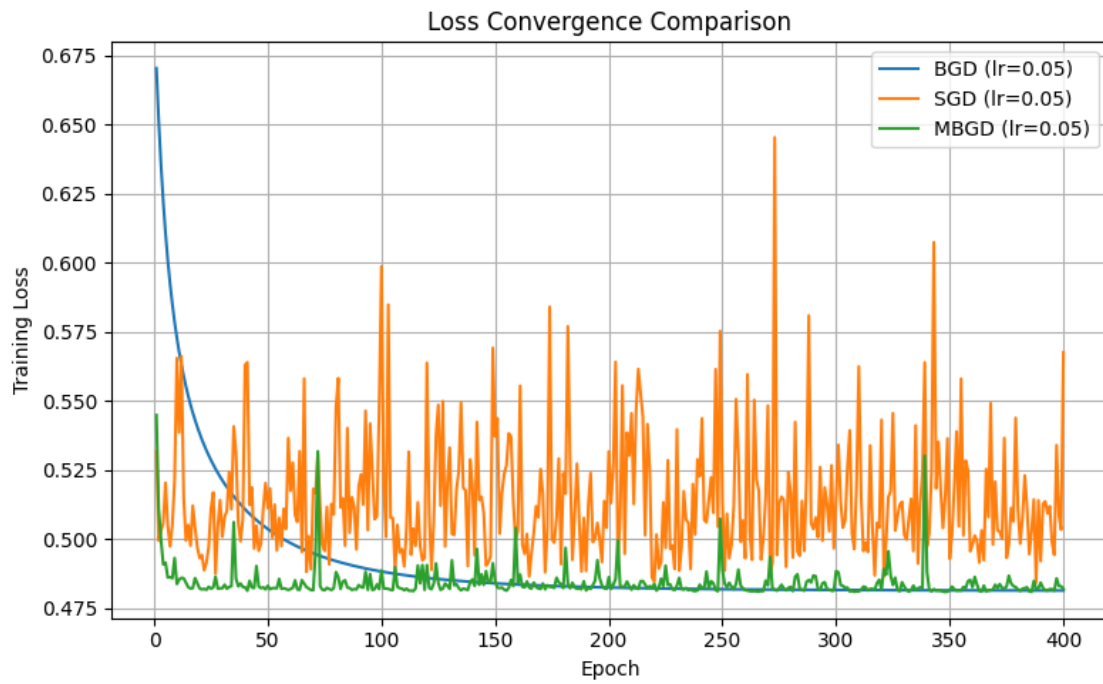
	optimizer	mean_final_test_acc	std_final_test_acc	mean_final_test_loss	\
0	bgd	0.746067	0.046734	0.480168	
1	mbgd	0.757303	0.039406	0.485645	
2	sgd	0.748315	0.049362	0.532157	

std_final_test_loss mean_epoch_time

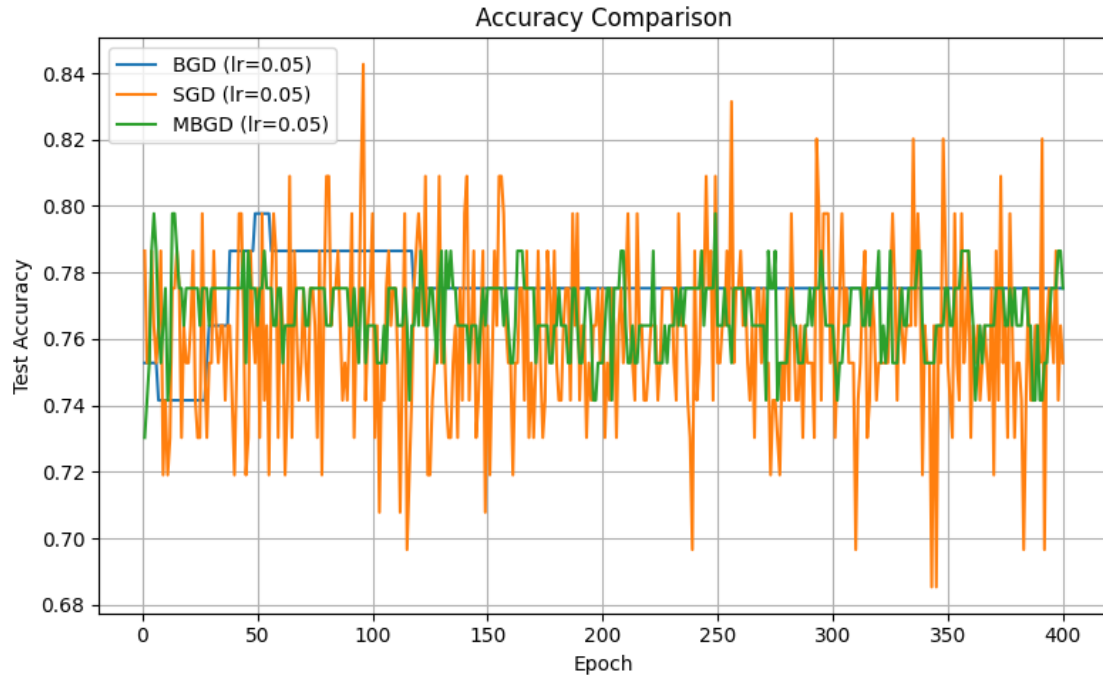
0	0.040764	0.000156
1	0.045015	0.000459
2	0.032654	0.004651

1.10 9. Plot convergence and accuracy curves

```
[10]: plot_loss_curves(histories)
```



```
[11]: plot_accuracy_curves(histories)
```



1.11 9a. Normalized convergence analysis: Updates and wall-clock time

The epoch-based plots above treat each optimizer equally despite their different update frequencies. BGD makes 1 parameter update per epoch, while SGD makes $n_samples$ updates per epoch, and MBGD makes $\text{ceil}(n_samples / \text{batch_size})$ updates. This section normalizes the convergence curves by plotting against cumulative parameter updates and cumulative wall-clock time to enable a more rigorous comparison.

```
[12]: def compute_updates_per_epoch(n_samples, optimizer, batch_size=16):
    """
    Compute the number of parameter updates for each optimizer per epoch.

    BGD: 1 update per epoch (full batch)
    SGD: n_samples updates per epoch (1 sample per batch)
    MBGD: ceil(n_samples / batch_size) updates per epoch
    """
    if optimizer == "bgd":
        return 1
    elif optimizer == "sgd":
        return n_samples
    elif optimizer == "mbgd":
        return int(np.ceil(n_samples / batch_size))
    else:
        raise ValueError(f"Unknown optimizer: {optimizer}")
```

```

def add_cumulative_metrics(history, n_samples, batch_size=16):
    """
    Add cumulative parameter updates and cumulative wall-clock time to history.
    """
    optimizer = history["optimizer"]
    updates_per_epoch = compute_updates_per_epoch(n_samples, optimizer,
    ↪batch_size)

    cumulative_updates = []
    cumulative_time = []

    total_updates = 0
    total_time = 0.0

    for i in range(len(history["epoch"])):
        total_updates += updates_per_epoch
        total_time += history["epoch_time"][i]
        cumulative_updates.append(total_updates)
        cumulative_time.append(total_time)

    history["cumulative_updates"] = cumulative_updates
    history["cumulative_time"] = cumulative_time
    history["updates_per_epoch"] = updates_per_epoch

# Add cumulative metrics to the last run's histories
n_train_samples = X_train.shape[0]
batch_sizes = {"bgd": 16, "sgd": 16, "mbgd": 16}

for h in histories:
    add_cumulative_metrics(h, n_train_samples, batch_sizes.get(h["optimizer"],
    ↪16))

```

```

[13]: def plot_loss_by_updates(histories):
    """Plot training loss convergence vs cumulative parameter updates."""
    plt.figure(figsize=(8, 5))
    for h in histories:
        label = f"{h['optimizer'].upper()} (lr={h['learning_rate']},
    ↪{h['updates_per_epoch']} updates/epoch)"
        plt.plot(h["cumulative_updates"], h["train_loss"], marker='o',
    ↪label=label, alpha=0.7)
    plt.xlabel("Cumulative Parameter Updates")
    plt.ylabel("Training Loss")
    plt.title("Loss Convergence vs Parameter Updates")
    plt.legend()
    plt.grid(True)

```

```

plt.tight_layout()
plt.show()

def plot_loss_by_time(histories):
    """Plot training loss convergence vs cumulative wall-clock time."""
    plt.figure(figsize=(8, 5))
    for h in histories:
        label = f"{h['optimizer'].upper()} (lr={h['learning_rate']})"
        plt.plot(h["cumulative_time"], h["train_loss"], marker='o',
↪label=label, alpha=0.7)
    plt.xlabel("Cumulative Wall-Clock Time (seconds)")
    plt.ylabel("Training Loss")
    plt.title("Loss Convergence vs Wall-Clock Time")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_accuracy_by_updates(histories):
    """Plot test accuracy convergence vs cumulative parameter updates."""
    plt.figure(figsize=(8, 5))
    for h in histories:
        label = f"{h['optimizer'].upper()} (lr={h['learning_rate']},
↪{h['updates_per_epoch']} updates/epoch)"
        plt.plot(h["cumulative_updates"], h["test_acc"], marker='o',
↪label=label, alpha=0.7)
    plt.xlabel("Cumulative Parameter Updates")
    plt.ylabel("Test Accuracy")
    plt.title("Accuracy Convergence vs Parameter Updates")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_accuracy_by_time(histories):
    """Plot test accuracy convergence vs cumulative wall-clock time."""
    plt.figure(figsize=(8, 5))
    for h in histories:
        label = f"{h['optimizer'].upper()} (lr={h['learning_rate']})"
        plt.plot(h["cumulative_time"], h["test_acc"], marker='o', label=label,
↪alpha=0.7)
    plt.xlabel("Cumulative Wall-Clock Time (seconds)")
    plt.ylabel("Test Accuracy")
    plt.title("Accuracy Convergence vs Wall-Clock Time")

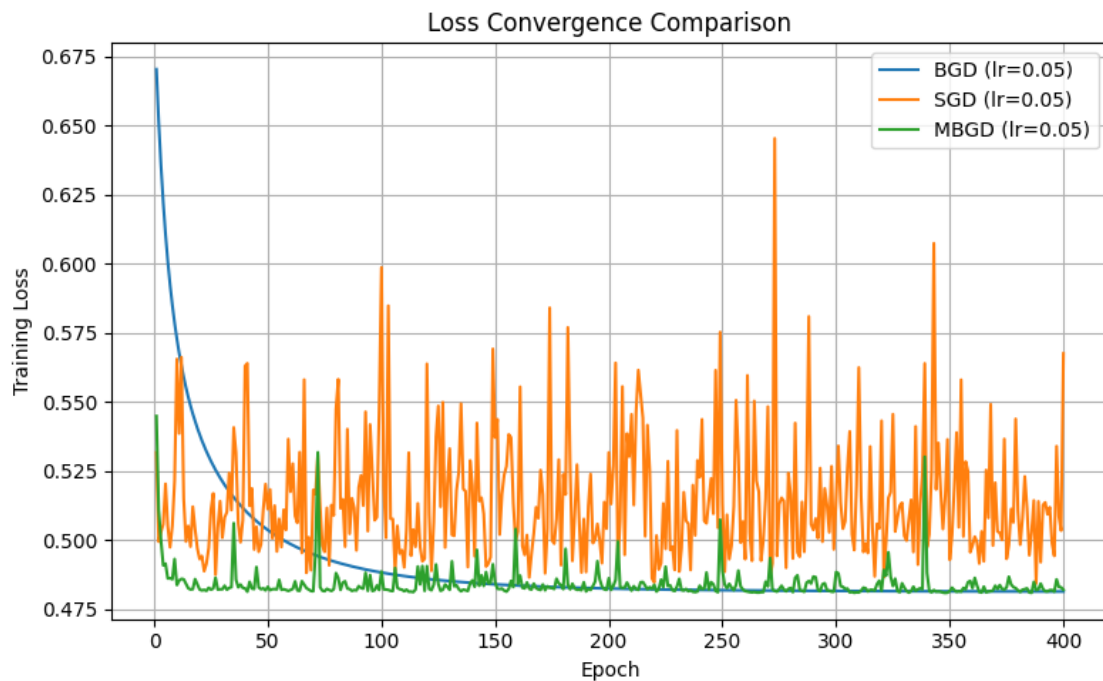
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

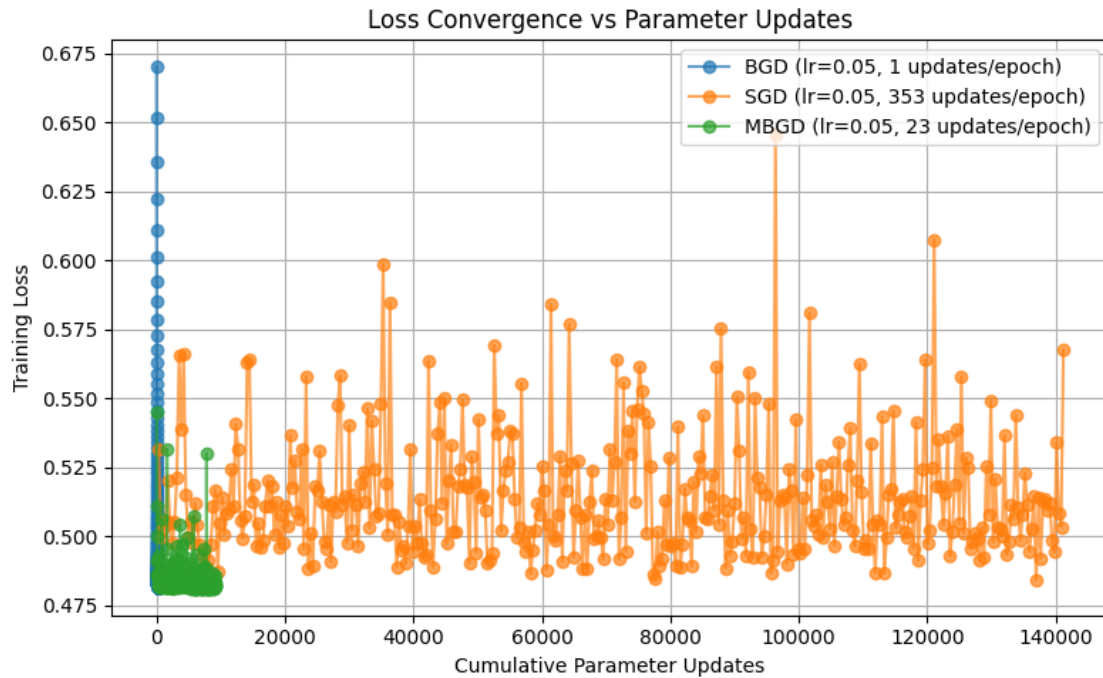
1.11.1 Loss convergence: three perspectives

Below we compare the three convergence representations for the same final seed's training run.

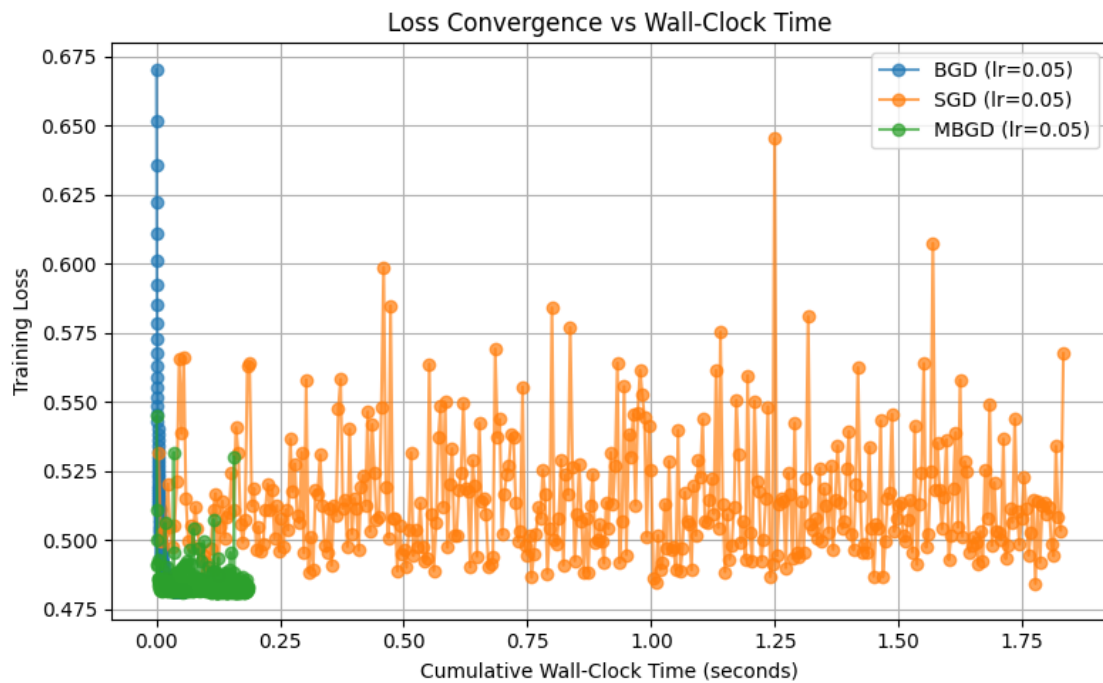
```
[14]: plot_loss_curves(histories) # epoch-based (original)
```



```
[15]: plot_loss_by_updates(histories) # update-based (normalized by parameter
    ↪ updates)
```

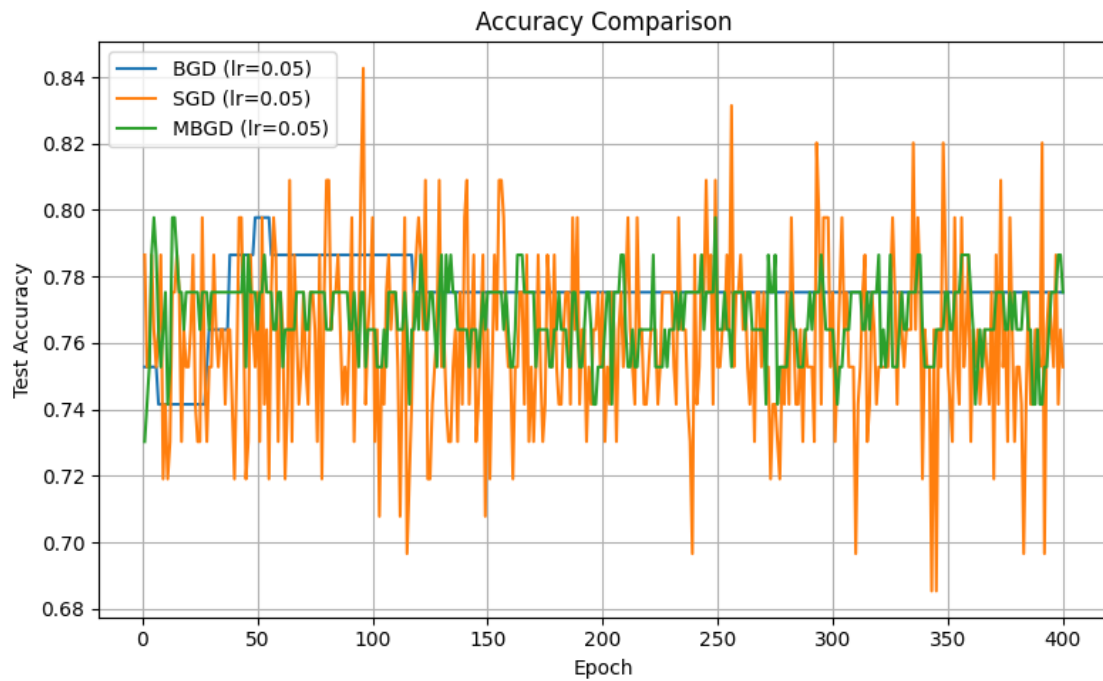


```
[16]: plot_loss_by_time(histories) # time-based (normalized by wall-clock time)
```

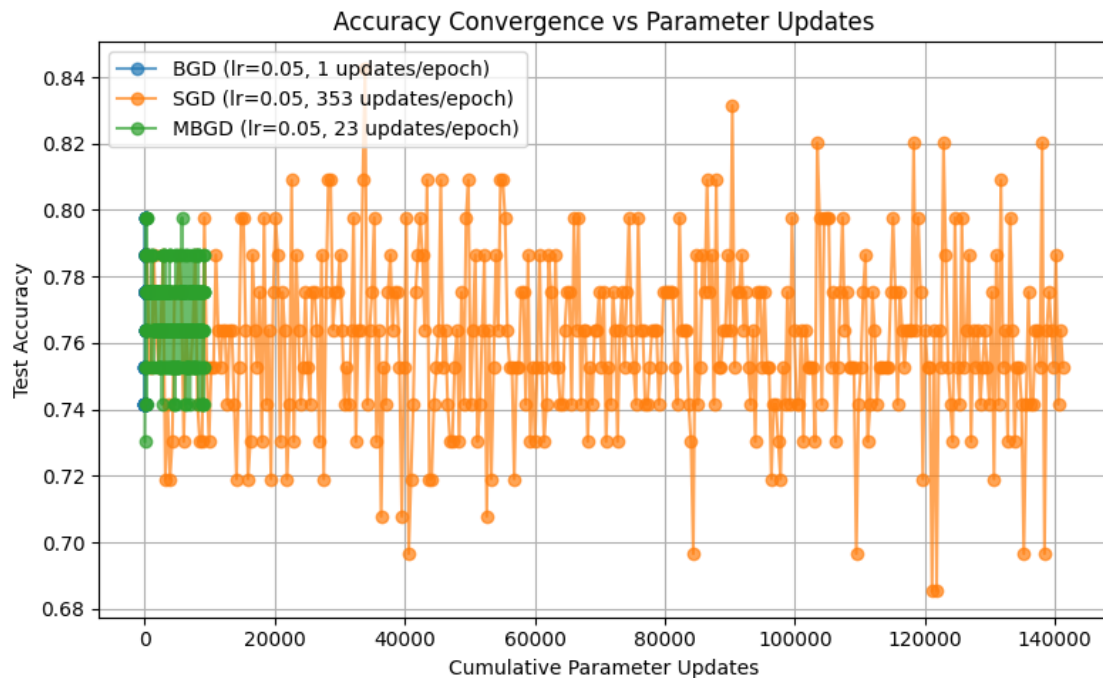


1.11.2 Accuracy convergence: three perspectives

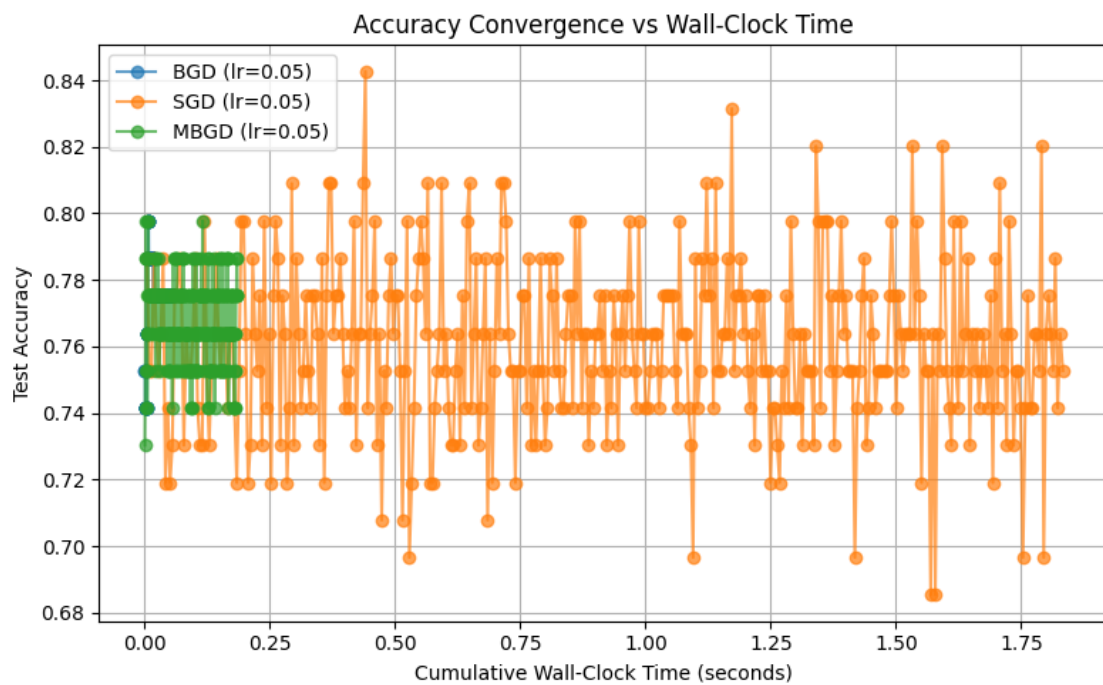
```
[17]: plot_accuracy_curves(histories)  # epoch-based (original)
```



```
[18]: plot_accuracy_by_updates(histories)  # update-based (normalized by parameter_↪updates)
```



```
[19]: plot_accuracy_by_time(histories) # time-based (normalized by wall-clock time)
```



1.11.3 Interpretation: Normalized convergence analysis

The three representations above reveal important differences in how the optimizers compare when normalized for computational effort:

- **Epoch-based plot:** Shows convergence speed in wall-clock epochs. SGD appears fastest here because it makes many updates per epoch.
- **Update-based plot:** Normalizes by parameter updates. This reveals how efficiently each optimizer uses its available updates to reduce loss/improve accuracy.
- **Time-based plot:** Shows real computational cost in seconds. This accounts for both the number of updates and the per-update overhead, revealing practical efficiency on the hardware.

For stochastic optimizers (SGD, MBGD), the wall-clock time perspective is often most relevant because it directly reflects the training cost incurred. The update-based perspective shows whether a method converges quickly *per update* when computational overhead is abstracted away.

1.12 9b. Variability across repeated trials

The repeated trials (5 seeds) enable measurement of training stability for each optimizer. Below we examine how much variance each optimizer exhibits due to stochastic effects and random initialization.

```
[20]: print("\n" + "="*80)
print("REPEATED-TRIALS SUMMARY: Mean ± Std Dev across 5 random seeds")
print("="*80)
print(summary_df.to_string(index=False))
print("\nInterpretation:")
print("- BGD typically shows low variance (deterministic updates on full batch).
↪")
print("- SGD and MBGD show higher variance due to stochastic batch sampling.")
print("- Standard deviation in test accuracy indicates training stability.")
print("- High std_final_test_loss or std_final_test_acc suggests sensitivity to
↪initialization/randomness.")
```

```
=====
REPEATED-TRIALS SUMMARY: Mean ± Std Dev across 5 random seeds
=====
```

optimizer	mean_final_test_acc	std_final_test_acc	mean_final_test_loss
bgd	0.746067	0.046734	0.480168
0.040764	0.000156		
mbgd	0.757303	0.039406	0.485645
0.045015	0.000459		
sgd	0.748315	0.049362	0.532157
0.032654	0.004651		

Interpretation:

- BGD typically shows low variance (deterministic updates on full batch).

- SGD and MBGD show higher variance due to stochastic batch sampling.
- Standard deviation in test accuracy indicates training stability.
- High std_final_test_loss or std_final_test_acc suggests sensitivity to initialization/randomness.

```
[21]: import matplotlib.pyplot as plt

# Visualize distribution of test accuracy across seeds for each optimizer
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

for idx, optimizer in enumerate(['bgd', 'sgd', 'mbgd']):
    ax = axes[idx]
    subset = df[df['optimizer'] == optimizer]
    ax.boxplot([subset['final_test_acc'].values], labels=[optimizer.upper()])
    ax.scatter([1] * len(subset), subset['final_test_acc'], alpha=0.5, s=60)
    ax.set_ylabel('Final Test Accuracy')
    ax.set_title(f'{optimizer.upper()}: Test Accuracy Distribution')
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nTest Accuracy Distribution by Optimizer:")
print(df.groupby('optimizer')['final_test_acc'].describe().round(4))
```

/tmp/ipykernel_125460/2952159138.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

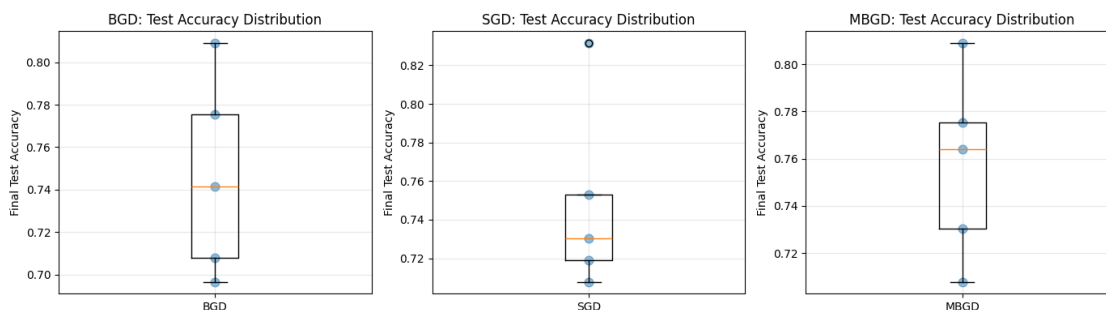
```
ax.boxplot([subset['final_test_acc'].values], labels=[optimizer.upper()])
```

/tmp/ipykernel_125460/2952159138.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot([subset['final_test_acc'].values], labels=[optimizer.upper()])
```

/tmp/ipykernel_125460/2952159138.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot([subset['final_test_acc'].values], labels=[optimizer.upper()])
```



Test Accuracy Distribution by Optimizer:

	count	mean	std	min	25%	50%	75%	max
optimizer								
bgd	5.0	0.7461	0.0467	0.6966	0.7079	0.7416	0.7753	0.8090
mbgd	5.0	0.7573	0.0394	0.7079	0.7303	0.7640	0.7753	0.8090
sgd	5.0	0.7483	0.0494	0.7079	0.7191	0.7303	0.7528	0.8315

1.13 10. Learning-rate sensitivity experiment

This section runs a small hyperparameter sweep to show how each optimizer reacts to different learning rates. The sweep is updated to cover a wider range for all three optimizers.

```
[22]: lr_grid = {
    'bgd': [0.001, 0.005, 0.01, 0.05, 0.1, 0.2],
    'sgd': [0.001, 0.005, 0.01, 0.05, 0.1, 0.2],
    'mbgd': [0.001, 0.005, 0.01, 0.05, 0.1, 0.2],
}

sweep_rows = []

for optimizer, lr_list in lr_grid.items():
    for lr in lr_list:
        _, history = train_model(
            X_train,
            y_train,
            X_test,
            y_test,
            optimizer=optimizer,
            learning_rate=lr,
            epochs=200,
            batch_size=16 if optimizer != 'sgd' else 1,
        )
        sweep_rows.append({
            'optimizer': optimizer,
            'learning_rate': lr,
            'final_train_loss': history['train_loss'][-1],
            'final_test_loss': history['test_loss'][-1],
            'final_train_acc': history['train_acc'][-1],
            'final_test_acc': history['test_acc'][-1],
            'avg_epoch_time': np.mean(history['epoch_time']),
        })

sweep_df = pd.DataFrame(sweep_rows)
sweep_df = sweep_df.sort_values(['optimizer', 'learning_rate']).
    ↪reset_index(drop=True)
```

```

sweep_df

best_idx = sweep_df.groupby('optimizer')['final_test_acc'].idxmax()
best_settings_df = sweep_df.loc[best_idx].sort_values('optimizer').
    ↪reset_index(drop=True)

print('Best learning rate per optimizer based on final test accuracy:\n')
print(best_settings_df[['optimizer', 'learning_rate', 'final_test_acc',
    ↪'final_train_acc']].to_string(index=False))

best_histories = []
for _, row in best_settings_df.iterrows():
    optimizer = row['optimizer']
    learning_rate = row['learning_rate']
    batch_size = 1 if optimizer == 'sgd' else 16
    _, history = train_model(
        X_train,
        y_train,
        X_test,
        y_test,
        optimizer=optimizer,
        learning_rate=learning_rate,
        epochs=400,
        batch_size=batch_size,
    )
    best_histories.append(history)

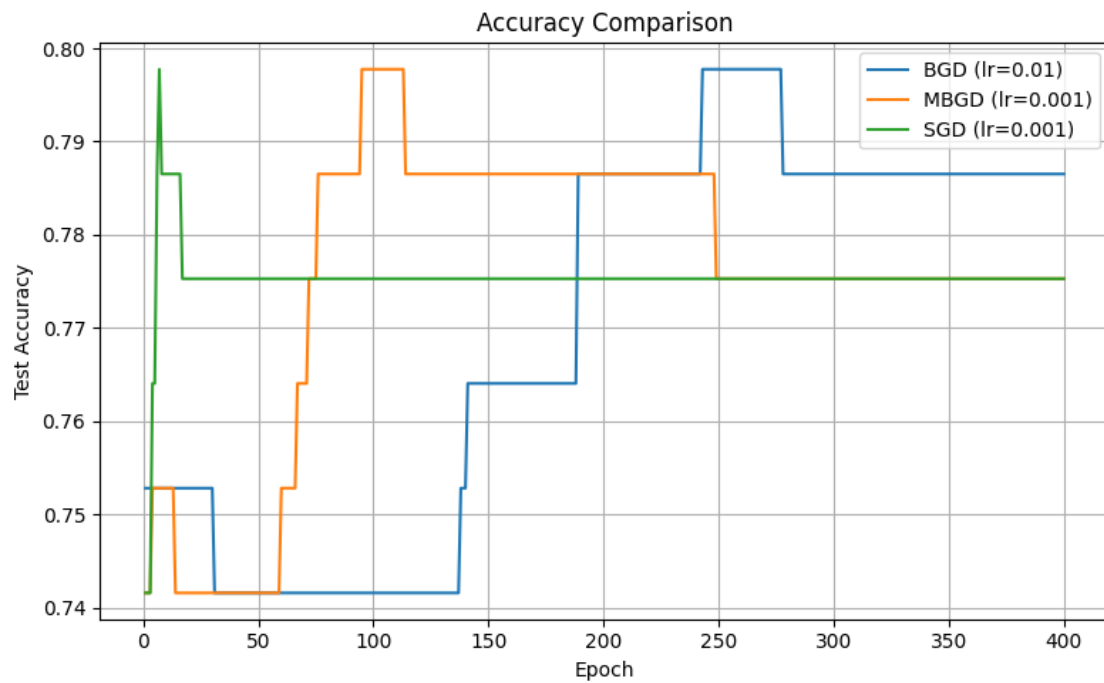
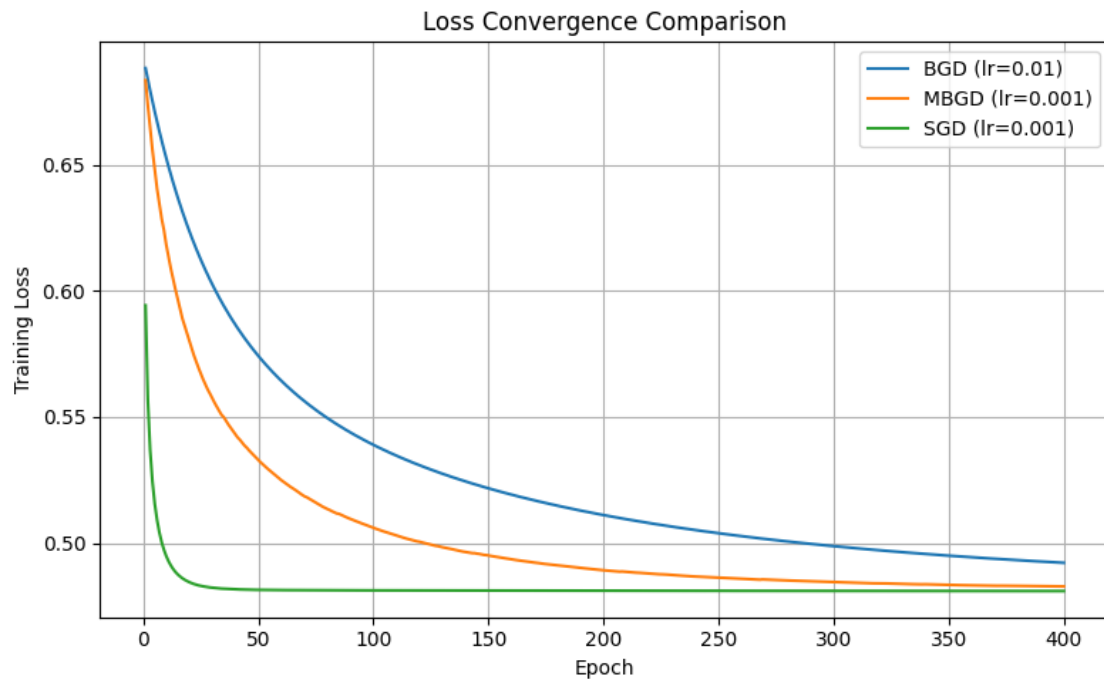
print('\nRetrained models using the best learning rate for each optimizer.')
plot_loss_curves(best_histories)
plot_accuracy_curves(best_histories)

```

Best learning rate per optimizer based on final test accuracy:

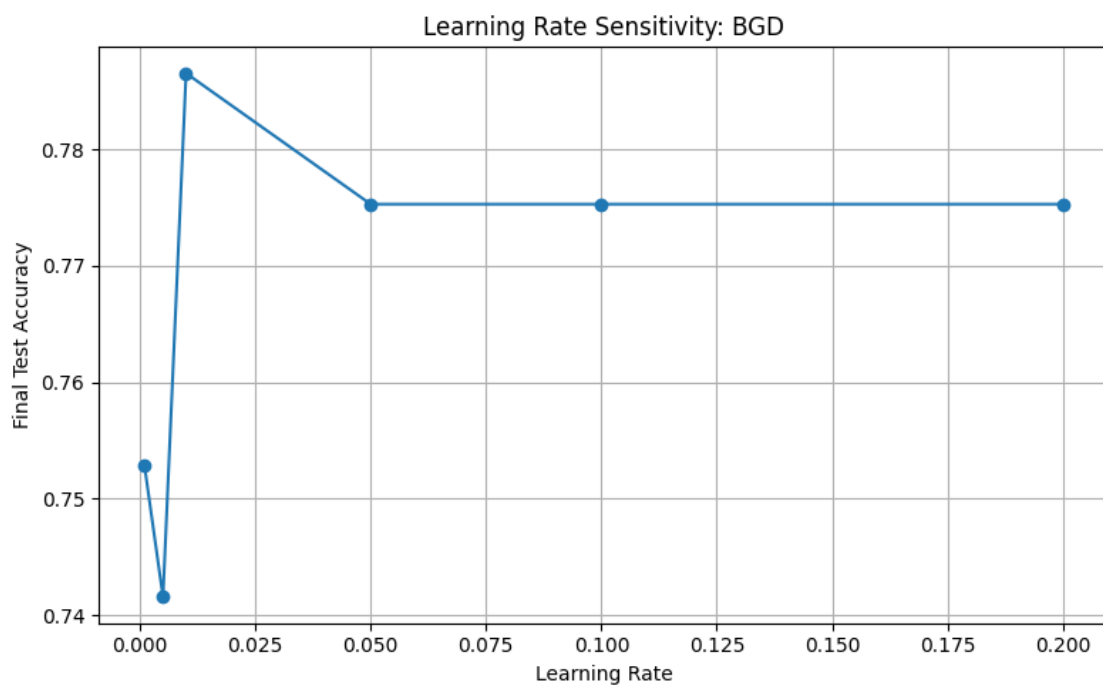
optimizer	learning_rate	final_test_acc	final_train_acc
bgd	0.010	0.786517	0.747875
mbgd	0.001	0.786517	0.753541
sgd	0.001	0.775281	0.739377

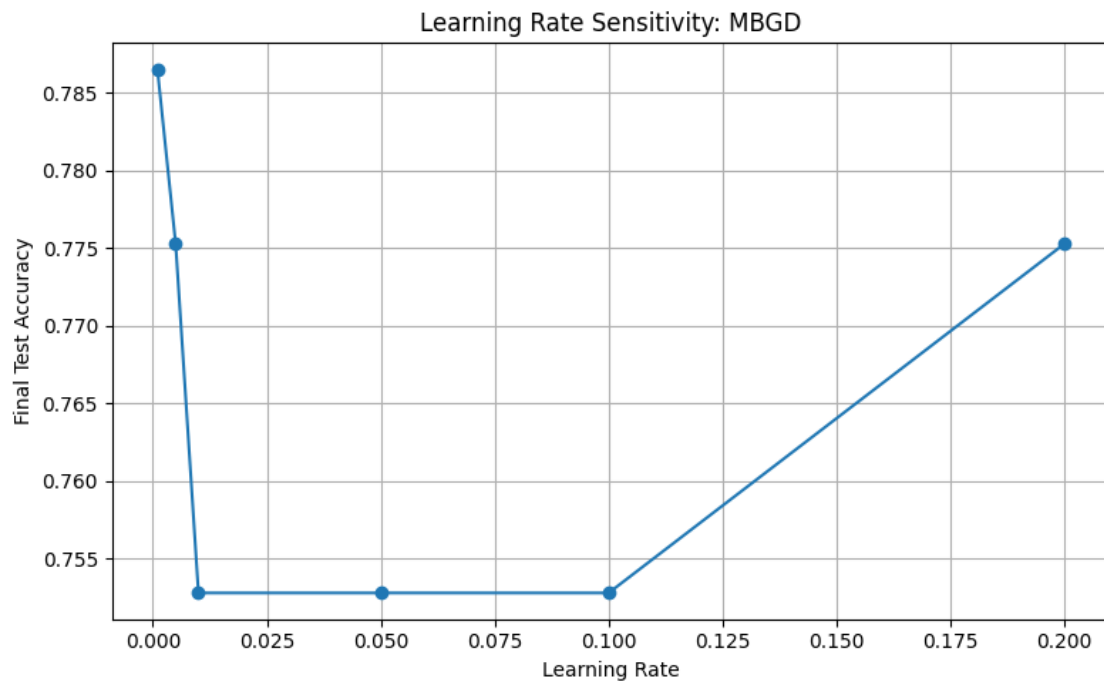
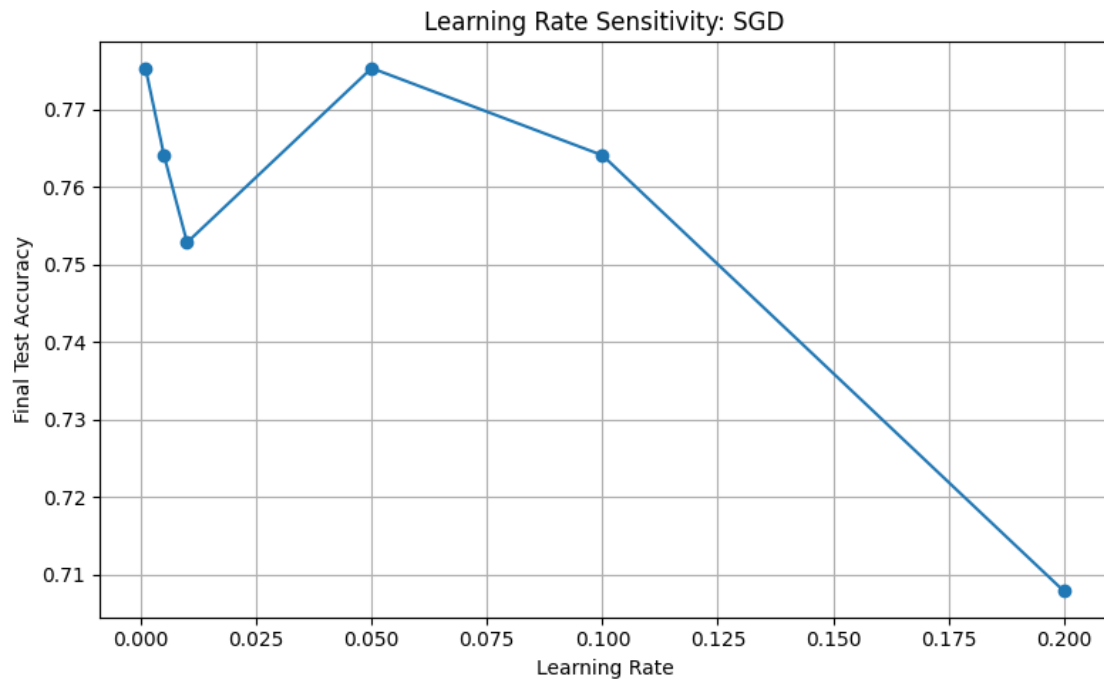
Retrained models using the best learning rate for each optimizer.



1.14 11. Visualize test accuracy across learning rates

```
[23]: for optimizer in ['bgd', 'sgd', 'mbgd']:
        subset = sweep_df[sweep_df['optimizer'] == optimizer].
        ↪sort_values('learning_rate')
        plt.figure()
        plt.plot(subset['learning_rate'], subset['final_test_acc'], marker='o')
        plt.xlabel('Learning Rate')
        plt.ylabel('Final Test Accuracy')
        plt.title(f'Learning Rate Sensitivity: {optimizer.upper()}')
        plt.grid(True)
        plt.tight_layout()
        plt.show()
```





1.15 12. Performance Boost Strategies

To address underfitting and increase the testing accuracy from the initial training algorithm, further fine-tuning is needed. Here are the main strategies to boost the performance: - Create new features by multiplying existing ones (e.g. BMI x BP), this allows the Logistic Regression Model to capture non-linear relationships without changing the algorithms.

```
[60]: def load_data(test_size=0.2, random_state=42):
    diabetes = load_diabetes(scaled=False, as_frame=True)
    df = diabetes.data.round(3)

    # # --- FEATURE ENGINEERING START ---
    # # Example: Add BMI and BP interaction
    # df['bmi_bp'] = df['bmi'] * df['bp']
    # # Example: Square the BMI
    # df['bmi_2'] = df['bmi'] ** 2
    # df['age_bins'] = pd.cut(df['age'], bins=[0, 30, 45, 60, 100], labels=[0, 1, 2, 3])
    # # --- FEATURE ENGINEERING END ---

    X = df.values
    y = (diabetes.target > diabetes.target.median()).astype(int).values

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Scaling is CRITICAL after engineering because  $x^2$  values
    # will be much larger than original values.
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    return X_train, X_test, y_train, y_test

seeds = [0, 1, 2, 3, 4] # repeat trials with these random seeds

settings = [
    {"optimizer": "bgd", "learning_rate": 0.05, "epochs": 800, "batch_size": 32},
    {"optimizer": "sgd", "learning_rate": 0.05, "epochs": 800, "batch_size": 32},
    {"optimizer": "mbgd", "learning_rate": 0.05, "epochs": 800, "batch_size": 32},
]
```



```

results = []
last_histories = {}

for seed in seeds:
    # ensure reproducible split / RNG for each trial
    np.random.seed(seed)

    X_train, X_test, y_train, y_test = load_data(test_size=0.2,
↪random_state=seed)

    for cfg in settings:
        _, history = train_model(
            X_train,
            y_train,
            X_test,
            y_test,
            optimizer=cfg["optimizer"],
            learning_rate=cfg["learning_rate"],
            epochs=cfg["epochs"],
            batch_size=cfg["batch_size"],
        )

        results.append(
            {
                "seed": seed,
                "optimizer": cfg["optimizer"],
                "final_train_loss": history["train_loss"][-1],
                "final_test_loss": history["test_loss"][-1],
                "final_train_acc": history["train_acc"][-1],
                "final_test_acc": history["test_acc"][-1],
                "avg_epoch_time": float(np.mean(history.get("epoch_time",
↪[0]))),
            }
        )

        # keep last history per optimizer for quick plotting
        last_histories[cfg["optimizer"]] = history

df = pd.DataFrame(results)
summary_df = (
    df.groupby("optimizer")
    .agg(
        mean_final_test_acc=("final_test_acc", "mean"),
        std_final_test_acc=("final_test_acc", "std"),
        mean_final_test_loss=("final_test_loss", "mean"),
        std_final_test_loss=("final_test_loss", "std"),

```

```

        mean_epoch_time=("avg_epoch_time", "mean"),
    )
    .reset_index()
)

print("\nRepeated-trials summary (mean ± std over seeds):\n")
print(summary_df)

# plot curves for the last seed run (one per optimizer)
histories = [last_histories[cfg["optimizer"]] for cfg in settings]

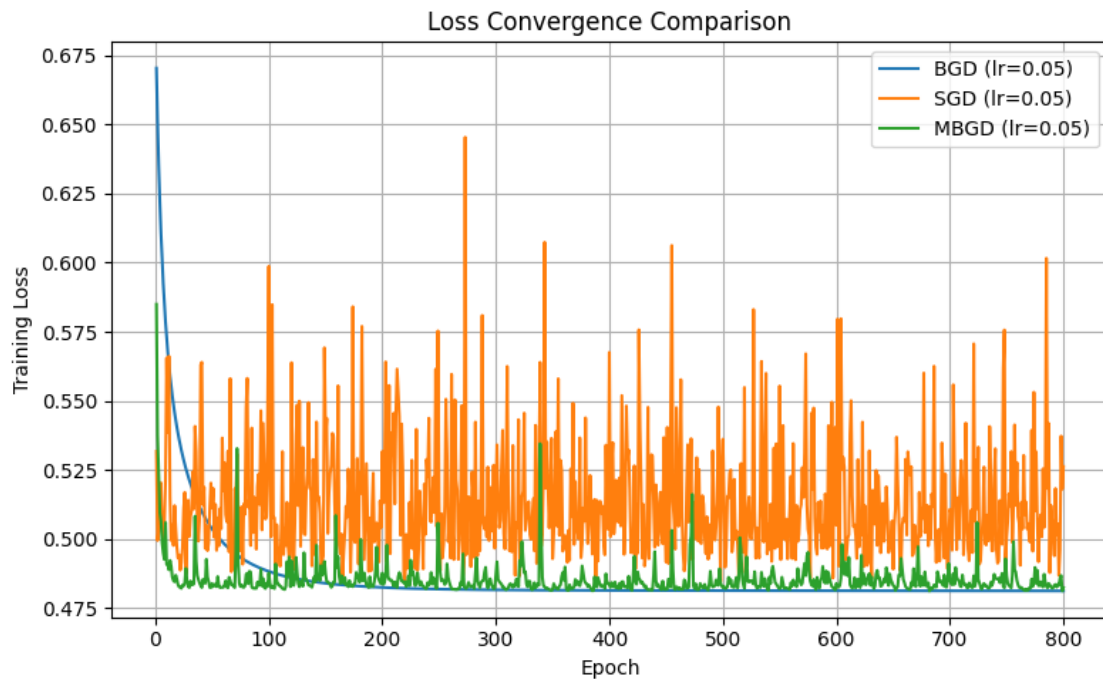
plot_loss_curves(histories)
plot_accuracy_curves(histories)

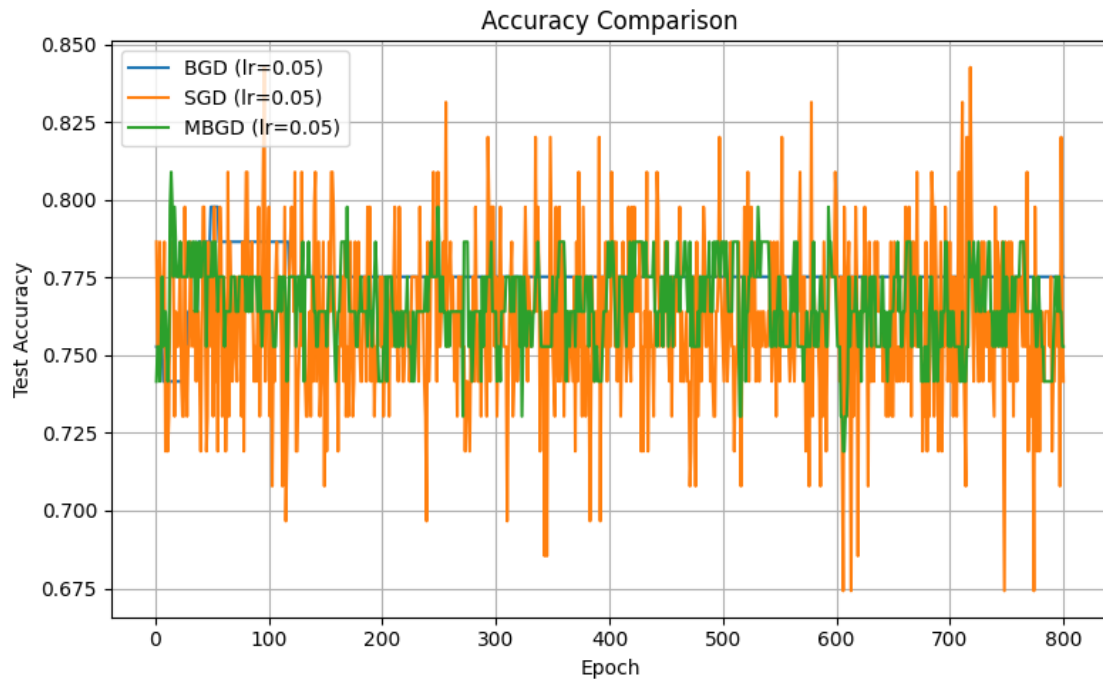
```

Repeated-trials summary (mean ± std over seeds):

	optimizer	mean_final_test_acc	std_final_test_acc	mean_final_test_loss	\
0	bgd	0.746067	0.046734	0.480696	
1	mbgd	0.741573	0.044236	0.483178	
2	sgd	0.748315	0.046734	0.522673	

	std_final_test_loss	mean_epoch_time
0	0.041957	0.000141
1	0.042792	0.000307
2	0.042690	0.004652





1.16 13. Observations

- All three optimization methods converge on the Iris dataset.
- SGD often reaches lower loss faster in epoch-based comparison because it updates parameters more frequently.
- BGD tends to be smoother but slower.
- MBGD usually provides a compromise between computational efficiency and predictive performance.
- Learning rate strongly affects both convergence speed and final accuracy.